CE152: Object-Oriented Programming

Encapsulation, Control Statements

Dr. Sebastian Halder

CSEE

2026

Encapsulation

Why do we worry about Encapsulation?

- *Encapsulation* is not about security (in the sense of encryption) but about control
- What we want to control is how variables in one class can be accessed by other classes
- We want to restrict access to certain variables (information hiding)
- It is *good practice* to use the must restrictive access level possible for any given variable or method

Coupling

- The reason is a concept called *coupling* (the dependence of one piece of code on another) which we want to minimise
- If there are many ways for class `A` to interact with class `B` there is a high chance that `A` will have to change if `B` changes
- Where the changes need to be made will also be more difficult to identify
- If there is just one way because we restrict the interaction as much as possible we can easily identify necessary changes or modify `B` in a way so that no changes in `A` are necessary at all!
- This is of course also possible without *encapsulation* (some languages like Python do not have access modifiers and support OOP)

Access modifiers in Java

- `public`: the member (variable or method) can be accessed from any class
- `private`: the member can only be accessed from the owning class
- `protected`: the member can be accessed from sub-classes and within the same package
- `default`: the member can only be accessed from classes in the same package

Let's make the age of Person private

- The age cannot be changed so we set it to private
- For now we are ok with anyone changing the name of the Person so we set it to public

```
class Person {
    private int age;
    public String name;

    Person(String name){
      this.name = name;
      this.age = 0;
    }
}
```

Other access modifiers

```
private int age;
protected float income;
public String name;
double weight;
```

Summary

- Access modifiers (`public`, `protected`, `private` and default) can be used to restrict how a class can be interacted with
- This is important to minimise *coupling* (the dependency of one class on another)
- Minimal *coupling* leads to easier maintenance of code

Refresher operators

Comparison operators

- There are several other operators we could use in Java

Operator	Meaning	true	false
==	equal	4 == 4	4 == 5
!=	unequal	4 != 5	4 != 4
<	less than	4 < 5	4 < 4
<=	less than or equal	4 <= 5	5 <= 4
>	greater than	5 > 4	5 > 5
>=	greater than or equal	5 >= 5	4 >= 5

Logical operators

- If we wanted to have two conditions we can combine statements with a logical operator:
  - Logical *and*: &&
  - Logical *or*: ||
  - Logical *not*: !
- We often use operators in conditional statements to decide which segment of code to execute during runtime

Conditionals

Using if statements

- An *if* statement executes the if-branch of the statement if the condition is true (we do this to execute this part of code only under those conditions)
- An *if* combined with an *else* statement executes either the if-branch (condition true) or the else-branch (condition false)
- We can use a combined *else if* to make the else statement conditional (see example later)

Example if

```
if (x < 0) {
  System.out.println("x is negative.");
}

if (x > 0) {
  System.out.println("x is positive.");
}
```

- Note: the {} are not needed if the segment of code following the `if` is only one line of code

Example if-else

```
if (x < 0) {
  System.out.println("x is negative.");
}
else {
  System.out.println("x is positive or zero.");
}
```

Example if-else-if

```
if (x < 0) {
  System.out.println("x is negative.");
}
else if (x > 0){
  System.out.println("x is positive.");
}
```

- We could add another `else`:

```
else {
  System.out.println("x is zero.");
}
```

The conditional operator

- This is a special syntax for defining a `if-then-else` statement:
- `variable x = (expression) ? value1 if true : value2 if false`
- Meaning: if the `expression` is true assign `value1` to `x` otherwise assign `value2` to `x`
- Many languages provide this syntax because it allows shorter code
- Examples:

```
int x = 5;
// if x is greater than zero assign 1 to y else assign -1
int y = (x > 0) ? 1 : -1;
```

Comparing Objects

- The logical operators do not function as we expect on Objects

```
String s1 = new String("CE152");
String s2 = new String("CE152");
System.out.println(s1==s2); // this will be false
System.out.println(s1.equals(s2)); // this will be true
```

- This is because the == operator compares the hashcode of the Objects

String comparisons

```java
String s1 = "oop";
String s2 = "oop";
System.out.println(s1==s2);
// this will be TRUE because Java uses the existing Object

String s3 = new String("oop");
String s4 = new String("oop");
System.out.println(s3 == s4);
// this will be FALSE because we manually create a new Object
```

Summary

- Conditionals exist in many programming languages
- They allow us to execute different segments of code depending on the value of certain variables
- We used:
  - `if` : segment is executed if the condition is true
  - `else`: follows an `if` and is always executed if the `if` statement is false
  - `else if`: also follows an `if` but is only executed if its own statement is true
- We also used a special syntax for defining `if-then-else` statements using the conditional operator:
  - `variable x = (expression) ? value1 if true : value2 if false`
- Remember Objects are not compared based on the content but the memory address
- Use dedicated methods such as `equals()` when comparing Objects

Loops

`for` loops

- We can use a `for` loop to repeat a segment of code multiple times
- A for loop has three statements in the *loop header*
  - initialisation
  - test condition
  - update condition
- The *loop body* is executed only if the test condition is true at least once
- The variable we initialise is only accessible in the scope of the *loop body*

for loop in Java

```
for(int i=0; i<10; i++)
{
  System.out.println("i = " + i);
}
```

Combined with if statment

```
for(int i=0; i<10; i++)
{
  if(i>5)
    System.out.println("i = " + i); // We can omit the {}
}
```

We can use other data types

```
for(double d=0.05; d<0.2; d+=0.01)
{
    System.out.println("d = " + d);
}
```

while loops

- A while loop has only one condition in the header
  - test condition
- We are responsible for creating the variables before hand and updating the variable in the loop body so that it will terminate at some point
- The advantage is now the variable will be accessible outside of the scope of the loop
- Again the body will never be executed if the test condition starts out false
- **If we do not set the correctly condition or update correctly the loop will run forever and the program will appear to be stuck or produce continuous output**

while loop in Java

```
int i = 0;
while (i<5){
  i++; // Again we could omit the {}
}
```

break

- We can also exit loops (for and while) using the keyword *break*

```
while (...){
  ...
  if (...)
    break;
  ...
}
```

- This will exit the loop, *not* execute the rest of the body
- As you see here it is not necessary to put curly brackets around single line condition or loop bodies (in this case the if statement)

continue

- We can jump to the next iteration of the loop (for and while) using the keyword *continue*

```
while (...){
  ...
  if (...)
    continue;
  ...
}
```

- This will *not* exit the loop, *not* execute the rest of the body and continue with the next iteration
- Do not overuse *break* and *continue* as they tend to make code harder to understand

Summary

- To execute a segment of code multiple times we can use `while` and `for` loops
- What to use when?
  - Often `for` loops are used when we know exactly how many times the code should run
  - If we do not know because it depends on user input or otherwise unknown conditions we use `while` loops
  - For example if you are waiting for something to be available (e.g. a file) you can run a `while` loop (with pauses!) until you can read the file and then exit the loop with `break`

Thank you!